



docker

DOCKER TRAINING ACTIVITIES

HANDS-ON GUIDE

Contents

Training preparation	2
Building and Running Docker Image and Container in Local Machine (Docker CLI)	7
Managing Docker components	8
Pushing and Pulling Images from Docker Hub	10
Dockerizing a Java Spring Boot App.....	12
Deploy the Dockerized Spring Boot application to Azure Container Instances (ACI) using GitHub Actions	15
Using Github Actions CI Pipeline with Amazon Linux	17
Docker Commands	20
Docker Volumes - Managing Persistent Data	23
Docker Networks - Managing Communication Between Containers	24
Running a Docker Container Interactively	26
Docker Storage Options	27
Volumes, Bind Mounts, and tmpfs Mounts	27
Using Docker Volumes for Data Persistence	28
Backing Up and Restoring Docker Volumes	28
Introduction to Docker Compose.....	29
Writing a docker-compose.yml File	29
Docker Compose Commands	30
Building and Running a Multi-Container Application	31
Multi-Stage Builds for Optimized Images in Docker	38
Docker Best Practices for Image Optimization and Security	40
Introduction to Docker Swarm Mode.....	42
Creating and Managing Docker Services in Swarm Mode	42
Scaling Containers Using Docker Swarm	43
Implementing User and Role-Based Access Control (RBAC)	43
Scanning Docker Images for Vulnerabilities	44

Docker Training Activities

Training preparation

1. Download and Install Git
2. Create GitHub Account
3. Download and prep Code Editor (ex. VSCode)
4. Download required SDK, Runtime Environment and Additional Services for your preferred language
Example:
 - Python
 - Java SDK and JRE
 - Apache/Nginx + PHP
5. Create Docker Hub Account
6. Download and install docker / docker desktop

Install Docker Desktop

- a) Download Docker Desktop
 - a. Visit the official Docker website: <https://www.docker.com/products/docker-desktop/>
 - b. Click Download for Windows.
- b) Run the Installer
 - c. Locate the downloaded .exe file and double-click to launch the installer.
 - d. Follow the installation wizard instructions.
- c) Enable WSL 2 (if prompted)
 - e. Docker Desktop requires Windows Subsystem for Linux 2 (WSL 2).
 - f. If WSL 2 is not installed, follow the prompts to enable it.
- d) Start Docker Desktop
 - g. Search for Docker Desktop in the Start menu and launch it.
 - h. Wait for it to initialize.
 - i. Accept the license agreement.
- e) Verify Installation

`docker --version`
- f) Expected Output:
Docker version XX.XX.XX, build XXXXXX
- g) You can also run:

`docker run hello-world`
- h) If Docker is working correctly, you will see a confirmation message.

Install Docker Engine on Ubuntu

- a) Update the package index and install dependencies

`sudo apt update`
`sudo apt install -y ca-certificates curl gnupg`

- b) Add Docker's official GPG key

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee
/etc/apt/keyrings/docker.asc > /dev/null
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

- c) Set up the Docker repository

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
```

- d) Install Docker Engine

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

- e) Verify Installation

```
sudo docker --version
```

- f) Expected Output:

Docker version XX.XX.XX, build XXXXXX

- g) Test Docker

```
sudo docker run hello-world
```

- h) This command runs a test container and confirms that Docker is working.

- i) To run Docker without sudo, add your user to the docker group:

```
sudo usermod -aG docker $USER
newgrp docker
```

- j) Then test:

```
docker run hello-world
```

Install Docker Engine on Amazon Linux 2023 using DNF

- a) Update the system

```
sudo dnf update -y
```

- b) Enable the Docker repository

```
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-
ce.repo
```

- c) Install Docker Engine

```
sudo dnf install -y docker-ce docker-ce-cli containerd.io
```

- d) Start and Enable Docker

```
sudo systemctl start docker
sudo systemctl enable docker
```

- e) Verify Installation

```
docker --version
```

- f) Expected Output:

Docker version XX.XX.XX, build XXXXXX

- g) Test Docker

```
sudo docker run hello-world
```

h) If everything is working, you should see the test message.

i) (Optional) Allow Docker to Run Without sudo

```
sudo usermod -aG docker $USER  
newgrp docker
```

j) Now test:

```
docker run hello-world
```

7. Prepare sample application to be dockerized

Sample Python App 1:

app.py

```
print("Hello, Docker!")
```

Dockerfile

```
FROM python:3.9-slim  
WORKDIR /app  
COPY app.py .  
CMD ["python", "app.py"]
```

Sample Python App 2:

app.py

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route('/')  
def home():  
    return "Hello, Flask and Docker!"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

Dockerfile

```
FROM python:3.9-slim  
WORKDIR /app  
COPY requirements.txt .  
RUN pip install -r requirements.txt  
COPY . .  
CMD ["python", "app.py"]
```

requirements.txt

```
Flask
```

Sample HTML App:

index.html

```
<!DOCTYPE html>
<html>
<head>  <title>Docker HTML</title>  </head>
<body>
  <h1>Hello, Dockerized HTML!</h1>
</body>
</html>
```

Dockerfile

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Sample PHP App:

index.php

```
<?php
  echo "Hello, Dockerized PHP!";
?>
```

Dockerfile

```
FROM php:8.0-apache
COPY index.php /var/www/html/
EXPOSE 80
```

Sample PHP & MySQL App:

index.php

```
<?php
$servername = "db";
$username = "root";
$password = "example";
$dbname = "testdb";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}

echo "Connected successfully to MySQL";
$conn->close();
?>
```

Dockerfile

```
FROM php:8.0-apache
RUN docker-php-ext-install mysqli
COPY index.php /var/www/html/
EXPOSE 80
```

docker-compose.yml

```
version: '3.8'
services:
  php:
    build: .
    ports:
      - "8080:80"
    depends_on:
      - db
  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: example
      MYSQL_DATABASE: testdb
```

The docker-compose.yml file is included in the Sample PHP and MySQL App example to simplify running multiple containers together:

1. Multi-Container Setup
 - ✓ The PHP application needs to connect to a MySQL database.
 - ✓ In a typical Docker setup, each service (PHP and MySQL) runs in its own container.
 - ✓ docker-compose.yml allows us to define and orchestrate multiple services easily.
2. Service Dependency Management
 - ✓ The depends_on key ensures that the PHP container waits for the MySQL database to start before running.
 - ✓ Without docker-compose, you'd have to manually start each container and manage their network connections.
3. Centralized Configuration
 - ✓ Instead of writing multiple docker run commands, all service configurations are defined in one place.
 - ✓ Example:
 - PHP container: Uses a Dockerfile to build the application.
 - MySQL container: Uses the mysql:5.7 image and environment variables to set up a database.
4. Simplified Networking
 - ✓ docker-compose automatically creates a shared network for all services.
 - ✓ The PHP container can refer to the database as db (instead of an IP address) since Docker Compose assigns meaningful names.

Notes

- If you only have a single PHP container, a Dockerfile alone is enough.
- If your application needs a database (like MySQL), docker-compose.yml makes it easier to manage the setup.

Building and Running Docker Image and Container in Local Machine (Docker CLI)

1. Sample Python App

Steps to Build and Run

```
cd /path/to/python-app
docker build -t python-app .
docker run --rm python-app
```

Expected Output

```
Hello, Docker!
```

2. Sample Flask App

Steps to Build and Run

```
cd /path/to/flask-app
docker build -t flask-app .
docker run -p 5000:5000 flask-app
```

Access the App

- Open a browser and go to <http://localhost:5000>
- Expected output: "Hello, Flask and Docker!"

3. Sample HTML Site

Steps to Build and Run

```
cd /path/to/html-site
docker build -t html-site .
docker run -p 8080:80 html-site
```

Access the Site

- Open a browser and go to <http://localhost:8080>
- Expected output: "Hello, Dockerized HTML!"

4. Sample PHP App

Steps to Build and Run

```
cd /path/to/php-app
docker build -t php-app .
docker run -p 8080:80 php-app
```


Access the PHP App

- Open a browser and go to <http://localhost:8080>
- Expected output: "Hello, Dockerized PHP!"

5. Sample PHP and MySQL App (Using Docker Compose)

Since this app consists of two containers (PHP & MySQL), we use docker-compose for easier management.

Navigate to the app directory

```
cd /path/to/php-mysql-app
docker-compose up -d
```

Access the PHP App

- Open a browser and go to <http://localhost:8080>
- Expected output: "Connected successfully to MySQL"

Stopping the Containers

```
docker-compose down
```

Managing Docker components

General Docker Commands

Command	Description
docker version	Show Docker version information.
docker info	Display system-wide information about Docker.
docker help	Show help for any Docker command.
docker stats	Show real-time resource usage of running containers.

Managing Containers

Starting, Stopping, and Running Containers

Command	Description
docker run image_name	Run a container from an image.
docker run -d image_name	Run a container in detached mode (background).
docker run -it image_name /bin/bash	Run a container interactively with a shell.
docker ps	Show running containers.
docker ps -a	Show all containers (including stopped ones).
docker stop container_id	Stop a running container.
docker start container_id	Start a stopped container.
docker restart container_id	Restart a container.
docker kill container_id	Forcefully stop a container.

Inspecting and Logging Containers

Command	Description
docker logs container_id	Show logs of a container.
docker inspect container_id	Show detailed information about a container.
docker top container_id	Show running processes inside a container.
docker exec -it container_id bash	Enter a running container with an interactive shell.
docker attach container_id	Attach to a running container's output.

Removing Containers

Command	Description
docker rm container_id	Remove a stopped container.
docker rm -f container_id	Force remove a running container.
docker rm \$(docker ps -aq)	Remove all stopped containers.

Managing Images

Listing and Pulling Images

Command	Description
docker images	Show all available images.
docker pull image_name	Download an image from Docker Hub.
docker rmi image_id	Remove an image.
docker rmi \$(docker images -q)	Remove all images.

Building and Tagging Images

Command	Description
docker build -t image_name .	Build an image from a Dockerfile.
docker tag image_name new_tag	Tag an existing image with a new name.
docker push image_name	Push an image to Docker Hub.

Managing Volumes

Command	Description
docker volume ls	List all Docker volumes.
docker volume create volume_name	Create a new volume.
docker volume inspect volume_name	Show volume details.
docker volume rm volume_name	Remove a volume.
docker volume prune	Remove all unused volumes.

Managing Networks

Command	Description
docker network ls	List all networks.
docker network create network_name	Create a new network.
docker network inspect network_name	Show network details.
docker network rm network_name	Remove a network.
docker network prune	Remove all unused networks.

Cleaning Up Unused Resources

Command	Description
<code>docker system prune</code>	Remove all stopped containers, unused networks, and dangling images.
<code>docker system df</code>	Show disk usage of Docker resources.
<code>docker system prune -a</code>	Remove all unused images (not just dangling ones).

Docker Compose Commands (For Multi-Container Apps)

Command	Description
<code>docker-compose up</code>	Start containers defined in <code>docker-compose.yml</code> .
<code>docker-compose up -d</code>	Start containers in detached mode.
<code>docker-compose down</code>	Stop and remove containers, networks, and volumes.
<code>docker-compose ps</code>	Show running containers.
<code>docker-compose logs</code>	Show logs of all services.
<code>docker-compose restart</code>	Restart all services.

Pushing and Pulling Images from Docker Hub

Docker Hub is a cloud-based registry where you can store and share Docker images. Below are the step-by-step instructions to push (upload) and pull (download) images from Docker Hub.

1. Pulling Images from Docker Hub

Docker Hub has a large collection of publicly available images. You can download (pull) them using the following steps.

Step 1: Search for an Image

If you want to pull an official image, visit <https://hub.docker.com/> and search for an image (e.g., `nginx`, `mysql`, `python`).

Alternatively, use this command to search from the terminal:

```
docker search ubuntu
```

Step 2: Pull an Image

To download an image, use:

```
docker pull ubuntu
```

*This pulls the latest version of Ubuntu.

To pull a specific version (tag):

```
docker pull ubuntu:20.04
```

Step 3: Verify the Pulled Image

After pulling, check if the image is available:

```
docker images
```

2. Pushing Images to Docker Hub

To push an image to Docker Hub, you need a Docker Hub account and must log in.

Step 1: Create a Docker Hub Account

- a) Go to <https://hub.docker.com/> and Sign up if you don't have an account.

Step 2: Log in to Docker Hub

```
docker login
```

Step 3: Tag the Image

Docker Hub requires images to be tagged with your username/repository name before pushing.

Example: If your Docker Hub username is myusername, tag the image like this:

```
docker tag ubuntu myusername/ubuntu-custom:1.0
```

Step 4: Push the Image

Push the tagged image to Docker Hub:

```
docker push myusername/ubuntu-custom:1.0
```

Step 5: Verify the Push

- a) Go to Docker Hub and check if your image appears in your repository.

3. Pulling Your Custom Image from Docker Hub

Once the image is pushed, anyone (with permissions) can pull it using:

```
docker pull myusername/ubuntu-custom:1.0
```

Now, they can run it:

```
docker run -it myusername/ubuntu-custom:1.0 bash
```

4. Deleting an Image from Docker Hub

- a) Go to Docker Hub and navigate to Repositories > Select the image.
- b) Click on Delete repository.

Summary of Key Commands

Command	Description
docker pull <image>	Download an image from Docker Hub.
docker pull <image>:<tag>	Download a specific version of an image.
docker login	Log in to Docker Hub.
docker tag <image> <username>/<repository>:<tag>	Tag an image for Docker Hub.
docker push <username>/<repository>:<tag>	Upload an image to Docker Hub.
docker images	List available images.

Dockerizing a Java Spring Boot App

Step 1: Create a Simple Spring Boot Application

- a) Go to Spring Initializr
- b) Select Maven project
- c) Choose Java as the language
- d) Set Spring Boot Version (e.g., 3.x.x)
- e) Add dependencies:
 - ✓ Spring Web (for REST API)
- f) Click Generate and download the project.
- g) Extract the downloaded ZIP and open the project in your favorite IDE (IntelliJ IDEA, VS Code, or Eclipse).

Step 2: Create a Simple REST Controller

Navigate to `src/main/java/com/example/demo` (assuming the package name is `com.example.demo`) and create a new controller.

Create `HelloController.java`

```
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, Dockerized Spring Boot!";
    }
}
```

Step 3: Build the Spring Boot Application

Before creating a Docker image, package the application into a JAR file.

Open a terminal in your project directory and run:

```
./mvnw clean package
```

*This will generate a JAR file inside the target directory (e.g., `target/demo-0.0.1-SNAPSHOT.jar`).

Step 4: Create a Dockerfile

In the root of your project, create a file named Dockerfile (no extension).

```
FROM openjdk:17-jdk-slim
```

```
WORKDIR /app
```

```
COPY target/demo-0.0.1-SNAPSHOT.jar app.jar
```

```
CMD ["java", "-jar", "app.jar"]
```

Step 5: Build the Docker Image

Run the following command in the terminal from the project root:

```
docker build -t spring-boot-docker .
```

```
docker images
```

Step 6: Run the Docker Container

Now, run the container using:

```
docker run -p 8080:8080 spring-boot-docker
```

Step 7: Test the Application

Open your browser or use curl to check the endpoint:

```
curl http://localhost:8080/api/hello
```

Expected Output:

Hello, Dockerized Spring Boot!

Step 8: Push the Image to Docker Hub

If you want to push the image to Docker Hub, follow these steps:

Login to Docker Hub (if not already logged in):

```
docker login
```

Tag the image with your Docker Hub username:

```
docker tag spring-boot-docker YOUR_DOCKERHUB_USERNAME/spring-boot-docker
```

Push the image to Docker Hub:

```
docker push YOUR_DOCKERHUB_USERNAME/spring-boot-docker
```

Now, you can pull and run your image from any machine using:

```
docker run -p 8080:8080 YOUR_DOCKERHUB_USERNAME/spring-boot-docker
```

Set Up GitHub Actions for Docker Deployment

1. Ensure you have a Docker Hub account.
2. Create a repository on GitHub and push your Spring Boot application.
3. Store Docker Hub credentials as GitHub Secrets:

4. Go to GitHub repository > Settings > Secrets and Variables > Actions > New repository secret.
5. Add the following secrets:

DOCKER_USERNAME: Your Docker Hub username.
DOCKER_PASSWORD: Your Docker Hub password or access token.

6. In your GitHub repository, create the following file:

.github/workflows/docker-deploy.yml

name: Build and Deploy to Docker Hub

on:

push:

branches:

- main

pull_request:

branches:

- main

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Checkout Repository

uses: actions/checkout@v4

- name: Set up JDK 17

uses: actions/setup-java@v3

with:

distribution: 'temurin'

java-version: '17'

- name: Build JAR File

run:

chmod +x ./mvnw

./mvnw clean package

- name: Log in to Docker Hub

uses: docker/login-action@v2

with:

username: \${ secrets.DOCKER_USERNAME }

password: \${ secrets.DOCKER_PASSWORD }

- name: Build Docker Image

run: |

```
docker build -t ${ secrets.DOCKER_USERNAME }}/spring-boot-docker:latest .
```

- name: Push Docker Image to Docker Hub

run: |

```
docker push ${ secrets.DOCKER_USERNAME }}/spring-boot-docker:latest
```

Deploy the Dockerized Spring Boot application to Azure Container Instances (ACI) using GitHub Actions

Step 1: Create an Azure Container Registry (ACR)

To store the Docker image in Azure, you need Azure Container Registry (ACR).

1. Log in to Azure CLI

az login

2. Create a Resource Group

az group create --name myResourceGroup --location eastus

3. Create an Azure Container Registry (ACR)

az acr create --resource-group myResourceGroup --name myacr1234 --sku Basic

*Replace myacr1234 with a unique name.

4. Log in to ACR

az acr login --name myacr1234

Step 2: Configure GitHub Secrets

In your GitHub repository, add the following Secrets:

AZURE_CLIENT_ID – Service principal ID (from Azure)

AZURE_CLIENT_SECRET – Service principal secret

AZURE_TENANT_ID – Azure tenant ID

AZURE_SUBSCRIPTION_ID – Azure subscription ID

ACR_NAME – Azure Container Registry name (e.g., myacr1234)

RESOURCE_GROUP – Azure Resource Group name (e.g., myResourceGroup)

CONTAINER_NAME – Name of the container in ACI

Create a Service Principal for GitHub Actions

Run the following command in Azure CLI:

```
az ad sp create-for-rbac --name "github-action-deploy" --role contributor \  
  --scopes /subscriptions/YOUR_SUBSCRIPTION_ID/resourceGroups/myResourceGroup \  
  --sdk-auth
```

*This will output a JSON object with clientId, clientSecret, tenantId, and subscriptionId. Copy these values and store them in GitHub Secrets.

Step 3: Create GitHub Actions Workflow

Create file: .github/workflows/azure-deploy.yml

```
name: Deploy to Azure Container Instances

on:
  push:
    branches:
      - main # Deploy when pushing to main branch

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      # Checkout code
      - name: Checkout repository
        uses: actions/checkout@v4

      # Log in to Azure
      - name: Log in to Azure
        uses: azure/login@v1
        with:
          client-id: ${ secrets.AZURE_CLIENT_ID }
          tenant-id: ${ secrets.AZURE_TENANT_ID }
          subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }

      # Log in to Azure Container Registry (ACR)
      - name: Log in to Azure Container Registry
        run: |
          az acr login --name ${ secrets.ACR_NAME }

      # Build and push Docker image to ACR
      - name: Build and push Docker image
        run: |
          docker build -t ${ secrets.ACR_NAME }.azurecr.io/spring-boot-app:latest .
          docker push ${ secrets.ACR_NAME }.azurecr.io/spring-boot-app:latest

      # Deploy container to Azure Container Instances (ACI)
      - name: Deploy to Azure Container Instances
        run: |
          az container create \
            --resource-group ${ secrets.RESOURCE_GROUP } \
            --name ${ secrets.CONTAINER_NAME } \
            --image ${ secrets.ACR_NAME }.azurecr.io/spring-boot-app:latest \
```

```
--dns-name-label spring-boot-app-${{ secrets.ACR_NAME }} \
--ports 8080 \
--registry-login-server ${{ secrets.ACR_NAME }}.azurecr.io \
--registry-username ${{ secrets.AZURE_CLIENT_ID }} \
--registry-password ${{ secrets.AZURE_CLIENT_SECRET }}
```

Verify Deployment

- name: Check Deployment Status

run: |

```
az container show --resource-group ${{ secrets.RESOURCE_GROUP }} --name ${{
secrets.CONTAINER_NAME }} --query "{FQDN:ipAddress.fqdn, Status:provisioningState}"
```

Step 4: Test the Deployment

After the workflow runs successfully, check the logs in GitHub Actions.

Find the FQDN (public URL) in the logs and test the app:

```
curl http://spring-boot-app-myacr1234.eastus.azurecontainer.io:8080/api/hello
```

Using Github Actions CI Pipeline with Amazon Linux

Sample CI/CD Workflow with Amazon Linux 2023, basic flask application, github, github actions, and docker engine

1. Setting Up Amazon Linux 2023

Step 1: Launch an Amazon Linux 2023 Instance

Step 2: Connect to the Instance

Use SSH to connect to your EC2 instance:

```
ssh -i your-key.pem ec2-user@your-ec2-public-ip
```

*Replace your-key.pem with your private key and your-ec2-public-ip with the instance's public IP.

Step 3: Update the System

Once logged in, update the system:

```
sudo dnf update -y
```

Step 4: Install Python and Dependencies

Install Python and necessary dependencies:

```
sudo dnf install python3 python3-pip python3-virtualenv -y
```

Verify installation:

```
python3 --version
pip3 --version
```

Step 5: Install Git

We need Git to work with GitHub:

```
sudo dnf install git -y
```

Verify installation:

```
git --version
```

Step 6: Install Docker

Docker will be required later for containerization:

```
sudo dnf install docker -y
```

Start and enable the Docker service:

```
sudo systemctl start docker  
sudo systemctl enable docker
```

Add the current user to the docker group to run Docker without sudo:

```
sudo usermod -aG docker ec2-user
```

Log out and log back in for the group changes to take effect, then verify:

```
docker --version
```

2. Creating a Basic Flask Application

Step 1: Set Up a Virtual Environment

Create a project directory and move into it:

```
mkdir flask-app && cd flask-app
```

Create a virtual environment:

```
python3 -m venv venv  
source venv/bin/activate
```

Step 2: Install Flask

```
pip install Flask
```

Step 3: Create a Simple Flask App

Create a file app.py and add the following code:

```
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route('/')  
def home():  
    return "Hello, Amazon Linux 2023!"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

Step 4: Run the Flask App

Run the app:

```
python app.py
```

*Visit <http://your-ec2-public-ip:5000> in a browser to see the output.

3. Version Control with GitHub

Now let's push the Flask app to GitHub.

Step 1: Initialize Git

Inside the flask-app directory:

```
git init
git config --global user.name "YourName"
git config --global user.email "youremail@example.com"
```

Step 2: Create a .gitignore File

```
echo "venv/" > .gitignore
echo "__pycache__/" >> .gitignore
```

Step 3: Commit and Push the Code

Create a new GitHub repository from GitHub's website.

Add the remote repository:

```
git remote add origin https://github.com/yourusername/flask-app.git
```

Add, commit, and push the files:

```
git add .
git commit -m "Initial commit"
git branch -M main
git push -u origin main
```

4. GitHub Actions for CI/CD

Step 1: Create a GitHub Actions Workflow

Inside the flask-app directory, create .github/workflows/main.yml:

```
name: Flask CI

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
```

```
uses: actions/checkout@v2
```

```
- name: Set up Python
  uses: actions/setup-python@v2
  with:
    python-version: "3.9"
```

```
- name: Install dependencies
  run: |
    python -m venv venv
    source venv/bin/activate
    pip install Flask
```

```
- name: Run tests
  run: python -m unittest discover
```

Push this workflow:

```
git add .github/workflows/main.yml
git commit -m "Add GitHub Actions workflow"
git push
```

*GitHub will now automatically run this workflow when you push code.

5. Dockerizing the Flask Application

Step 1: Create a Dockerfile

Create a Dockerfile in the project root:

```
FROM python:3.9-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Step 2: Build and Run Docker Image

```
sudo usermod -aG docker $USER          #then exit to close the session, reconnect afterwards
docker build -t flask-app .
docker run -p 5000:5000 flask-app
```

Docker Commands

Check Docker version:

```
docker --version
```

Check system-wide Docker info:

```
docker info
```

List Available Images

```
docker images
```

Pull an Image from Docker Hub

```
docker pull ubuntu
```

Build a Docker Image

```
docker build -t myapp .
```

(-t assigns a name/tag to the image)

Tag an Image

```
docker tag myapp myrepo/myapp:v1
```

Remove an Image

```
docker rmi myapp
```

Run a Container

```
docker run -d -p 8080:80 myapp
```

(-d runs it in the background, -p maps ports)

List Running Containers

```
docker ps
```

List All Containers (Including Stopped Ones)

```
docker ps -a
```

Stop a Running Container

```
docker stop container_id
```

Start a Stopped Container

```
docker start container_id
```

Remove a Container

```
docker rm container_id
```

Run an Interactive Shell Inside a Running Container

```
docker exec -it container_id /bin/bash
```

View Container Logs

```
docker logs container_id
```

Follow Logs in Real-Time

```
docker logs -f container_id
```

Inspect a Running Container

```
docker inspect container_id
```

Check Resource Usage

```
docker stats
```

List Docker Volumes

```
docker volume ls
```

Create a Named Volume

```
docker volume create myvolume
```

List Docker Networks

```
docker network ls
```

Connect a Container to a Network

```
docker network connect mynetwork container_id
```

Stop All Running Containers

```
docker stop $(docker ps -q)
```

Remove All Stopped Containers

```
docker rm $(docker ps -a -q)
```

Remove All Images

```
docker rmi $(docker images -q)
```

Remove All Docker Volumes

```
docker volume rm $(docker volume ls -q)
```

5.2 Upload docker image to Docker Hub

- a) Go to hub.docker.com, sign in or sign up
- b) Back to the machine with docker container, use the following command to login

```
docker login
```

- c) Tag the Docker Image
 - a. List your local Docker images:

```
docker images
```

b. You'll see an output like:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
flask-app	latest	a1b2c3d4e5f6	10 minutes ago	120MB

d) To push this image to Docker Hub, you need to tag it with your Docker Hub username:

```
docker tag flask-app your-dockerhub-username/flask-app:latest
```

e) For example, if your Docker Hub username is john123, run:

```
docker tag flask-app john123/flask-app:latest
```

f) Push the Image to Docker Hub

```
docker push your-dockerhub-username/flask-app:latest
```

g) Example:

```
docker push john123/flask-app:latest
```

h) Verify on Docker Hub, Go to Docker Hub.

i) Log in and check the Repositories tab to see your uploaded image.

j) Pull and Run from Any Machine, To pull the image on another machine:

```
docker pull your-dockerhub-username/flask-app:latest
```

k) Then, run it:

```
docker run -p 5000:5000 your-dockerhub-username/flask-app
```

Docker Volumes - Managing Persistent Data

Docker Volumes store data outside of the container, ensuring that it persists even when the container is removed.

Step 1: Create a Volume

```
docker volume create my_volume
```

Verify:

```
docker volume ls
```

Step 2: Use the Volume in a Container

Run a container and mount the volume:

```
docker run -d --name my_container -v my_volume:/data busybox sleep 3600
```

This mounts my_volume to /data inside the container.

The busybox container will sleep for 1 hour.

Step 3: Add Data to the Volume

Attach to the container:

```
docker exec -it my_container sh
```


Inside the container:

```
echo "Hello, Docker Volume!" > /data/test.txt  
exit
```

Step 4: Check the Data Persistence

Remove the container:

```
docker rm -f my_container
```

Run a new container with the same volume:

```
docker run -it --rm -v my_volume:/data busybox cat /data/test.txt
```

Output:

Hello, Docker Volume!

Step 5: Remove a Volume

First, stop and remove containers using the volume:

```
docker rm -f my_container
```

Now remove the volume:

```
docker volume rm my_volume
```

Summary of Volume Commands

Command	Description
docker volume create my_volume	Create a new volume
docker volume ls	List all volumes
docker run -v my_volume:/data	Mount a volume to a container
docker volume rm my_volume	Remove a volume
docker volume prune	Remove all unused volumes

Docker Networks - Managing Communication Between Containers

Docker Networks enable communication between multiple containers.

Types of Docker Networks

- Bridge (default) - Used for communication between containers on the same host.
- Host - Shares the host's network stack.
- None - No networking (isolated).
- Overlay - Used in Docker Swarm for multi-host communication.
- Macvlan - Assigns a MAC address to the container.

Example: Creating a Custom Bridge Network

Step 1: Create a Network

```
docker network create my_network  
# Verify:  
docker network ls
```

Step 2: Run Containers on the Custom Network

Start a simple web server (nginx) and connect it to the network:

```
docker run -d --name webserver --network my_network nginx
```

Start another container (busybox) on the same network:

```
docker run -it --rm --network my_network busybox sh
```

Inside the busybox shell, test connectivity:

```
ping webserver
```

Expected output:

```
64 bytes from webserver (XXX.XXX.XXX.XXX): icmp_seq=1 ttl=64 time=0.089 ms
```

*This confirms that the two containers can communicate.

Step 3: Inspect Network Details

```
docker network inspect my_network
```

Step 4: Remove Network

First, stop and remove containers:

```
docker rm -f webserver
```

Then remove the network:

```
docker network rm my_network
```

Summary of Network Commands

Command	Description
docker network create my_network	Create a new network
docker network ls	List all networks
docker network inspect my_network	Get details of a network
docker network rm my_network	Remove a network
docker run --network my_network	Connect a container to a network

Running Multi-Container App with Docker Compose

Instead of manually managing networks, Docker Compose simplifies multi-container applications.

Example: Running an Nginx and Alpine Client

Step 1: Create docker-compose.yml

```
version: '3'
services:
  web:
    image: nginx
    networks:
      - my_network
  alpine:
    image: alpine
    command: sleep 3600
    networks:
      - my_network
networks:
  my_network:
    driver: bridge
```

Step 2: Start the Multi-Container App

```
docker-compose up -d
```

Step 3: Test Communication

```
docker exec -it $(docker ps -q -f name=alpine) sh
ping web
```

Running a Docker Container Interactively

Run a Docker container interactively, you typically use the `-it` flags with the `docker run` command. Here's how you can do it:

```
docker run -it <image_name> /bin/bash
```

or

```
docker run -it <image_name> /bin/sh
```

*(depending on whether the container uses Bash or just has sh available.)

Examples

Run an interactive Ubuntu container

```
docker run -it ubuntu /bin/bash
```

- This pulls the ubuntu image if it's not available locally.
- It starts the container and opens an interactive Bash shell.

Run an interactive Alpine container

```
docker run -it alpine /bin/sh
```

*Since Alpine Linux is minimal, it uses /bin/sh instead of Bash.

Run an interactive container with a mounted volume

```
docker run -it -v $(pwd):/app ubuntu /bin/bash
```

*This mounts the current directory to /app inside the container.

Docker Storage Options

Docker provides multiple storage options to manage and persist data generated by containers. The three main options are:

- Volumes: Managed by Docker and stored in /var/lib/docker/volumes/. Best for data persistence.
- Bind Mounts: Maps a host directory to a container. Offers flexibility but less security.
- tmpfs Mounts: Stores data in RAM, providing speed but losing data when the container stops.

Volumes, Bind Mounts, and tmpfs Mounts

1. Docker Volumes (Recommended for persistent storage)
 - Created and managed by Docker.
 - Stored outside the container's filesystem.
 - Survives container deletion.

Create a volume:

```
docker volume create my_volume
```

Run a container using the volume:

```
docker run -it -v my_volume:/data ubuntu /bin/bash
```

Verify the volume exists:

```
docker volume ls
```

Check volume details:

```
docker volume inspect my_volume
```

2. Bind Mounts (Directly maps host directory to a container)
 - Allows accessing and modifying host files inside a container.
 - Less secure than volumes but useful for development.

Run a container with a bind mount:

```
docker run -it -v /path/on/host:/data ubuntu /bin/bash
```

Modify files inside /data in the container, and they reflect on the host.

3. tmpfs Mounts (For temporary in-memory storage)
 - Stored in RAM, very fast, but data is lost when the container stops.

Run a container with a tmpfs mount:

```
docker run -it --tmpfs /mytmpfs:size=100m ubuntu /bin/bash
```

*Any data stored in /mytmpfs will be lost once the container stops.

Using Docker Volumes for Data Persistence

To persist data across container restarts:

Create a volume

```
docker volume create persistent_data
```

Use the volume in a container

```
docker run -d --name my_container -v persistent_data:/data ubuntu
```

Write data inside the container

```
docker exec -it my_container /bin/bash
echo "Hello, Docker!" > /data/hello.txt
```

Stop and remove the container

```
docker stop my_container
docker rm my_container
```

Recreate the container and access data

```
docker run -it --rm -v persistent_data:/data ubuntu cat /data/hello.txt
```

The file hello.txt still exists, proving persistence.

Backing Up and Restoring Docker Volumes

To protect volume data, you can back up and restore it.

Backup a Volume

Create a backup container and mount the volume:

```
docker run --rm -v persistent_data:/data -v $(pwd):/backup ubuntu tar czf /backup/data_backup.tar.gz -C /data .
```

*This compresses /data inside persistent_data into data_backup.tar.gz in the current directory.

Restore a Volume

Create a new volume for restoration:

```
docker volume create restored_data
```

Restore the backup:

```
docker run --rm -v restored_data:/data -v $(pwd):/backup ubuntu tar xzf /backup/data_backup.tar.gz -C /data
```

Verify the restored volume:

```
docker run --rm -v restored_data:/data ubuntu ls /data
```

*The backed-up files should now be available in the restored_data volume.

Introduction to Docker Compose

What is Docker Compose?

Docker Compose is a tool that allows you to define and manage multi-container Docker applications using a YAML file. Instead of running multiple docker run commands manually, you can define all the services, networks, and volumes in a single docker-compose.yml file and start everything with one command.

Why Use Docker Compose?

- ✓ Simplifies multi-container management (e.g., Web + Database)
- ✓ Defines configurations in a single file (docker-compose.yml)
- ✓ Enables service scaling (e.g., docker-compose up --scale app=3)
- ✓ Works well with development and production environments

Writing a docker-compose.yml File

A docker-compose.yml file is a configuration file where you define services, networks, and volumes for your Docker application.

Basic Structure of docker-compose.yml

```
version: '3.8'
services:
  app:
    image: nginx
    ports:
      - "8080:80"
  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: mydatabase
```

Explanation:

- version: '3.8' → Specifies the Docker Compose file format version.
- services: → Defines the containers (app and db).
- app: → Runs an Nginx container, mapping port 8080 on the host to port 80 in the container.
- db: → Runs a MySQL container with a specified root password and database name.

Docker Compose Commands

Once you have a docker-compose.yml file, you can use the following commands:

1. Start Services

```
docker-compose up
```

- Starts all services defined in docker-compose.yml.
- If images aren't available locally, they will be pulled from Docker Hub.

2. Start in Detached Mode (Background)

```
docker-compose up -d
```

*Runs containers in the background.

3. View Running Services

```
docker-compose ps
```

*Lists all running services.

4. Stop Services

```
docker-compose down
```

*Stops and removes all services.

5. Restart Services

```
docker-compose restart
```

*Restarts all running services.

6. View Logs of a Service

```
docker-compose logs -f app
```

*Shows logs for the app service in real time.

7. Scale a Service

```
docker-compose up --scale app=3 -d
```

*Runs 3 instances of the app service.

8. Remove All Containers and Volumes

```
docker-compose down -v
```

*Removes containers, networks, and volumes.

Building and Running a Multi-Container Application

We will create a web application with a MySQL database using Docker Compose.

Example 1: PHP & MySQL App

Step 1: Create a Project Directory and add the following code to a new file (index.php)

index.php

```
<?php
$servername = "db";
$username = "root";
$password = "example";
$dbname = "testdb";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

echo "Connected successfully to MySQL";
$conn->close();
?>
```

Dockerfile

```
FROM php:8.0-apache
RUN docker-php-ext-install mysqli
COPY index.php /var/www/html/
EXPOSE 80
```

Step 2: Create a docker-compose.yml File

Create a file named docker-compose.yml and add the following content:

```
networks:
  my_custom_network:
    driver: bridge

services:
  web:
    image: php:apache
    container_name: web_server
    volumes:
      - ./www:/var/www/html
    networks:
      - my_custom_network
```



```
ports:
  - "8080:80"
depends_on:
  - db

db:
  image: mysql:5.7
  container_name: mysql_db
  restart: always
  environment:
    MYSQL_ROOT_PASSWORD: rootpassword
    MYSQL_DATABASE: mydatabase
    MYSQL_USER: user
    MYSQL_PASSWORD: userpassword
  networks:
    - my_custom_network
  volumes:
    - db_data:/var/lib/mysql

volumes:
  db_data:
```

Step 3: Start the Multi-Container Application

```
docker-compose up -d
```

- The web application runs on `http://localhost:8080`.
- The MySQL database container starts in the background.

Step 4: Verify Running Containers

```
docker-compose ps
```

Step 5: Stop and Remove the Containers

```
docker-compose down -v
```

*Stops all services and removes volumes.

The `docker-compose.yml` file is included in the Sample PHP and MySQL App example to simplify running multiple containers together:

a) Multi-Container Setup

- ✓ The PHP application needs to connect to a MySQL database.
- ✓ In a typical Docker setup, each service (PHP and MySQL) runs in its own container.
- ✓ `docker-compose.yml` allows us to define and orchestrate multiple services easily.

b) Service Dependency Management

- ✓ The `depends_on` key ensures that the PHP container waits for the MySQL database to start before running.
- ✓ Without `docker-compose`, you'd have to manually start each container and manage their network connections.

c) Centralized Configuration

- ✓ Instead of writing multiple docker run commands, all service configurations are defined in one place.
- ✓ Example:
 - PHP container: Uses a Dockerfile to build the application.
 - MySQL container: Uses the mysql:5.7 image and environment variables to set up a database.

d) Simplified Networking

- ✓ docker-compose automatically creates a shared network for all services.
- ✓ The PHP container can refer to the database as db (instead of an IP address) since Docker Compose assigns meaningful names.

Notes

- If you only have a single PHP container, a Dockerfile alone is enough.
- If your application needs a database (like MySQL), docker-compose.yml makes it easier to manage the setup.

Example 2: PHP and MySQL Web App with CRUD

Step 1: Setup Your Project Directory

```
php-mysql-docker/  
├─ app/  
│  ├─ index.php  
│  ├─ db.php  
│  ├─ create.php  
│  ├─ read.php  
│  ├─ update.php  
│  └─ delete.php  
├─ docker-compose.yml  
├─ Dockerfile  
├─ .env  
└─ init.sql
```

Step 2: Create docker-compose.yml

This file defines services for PHP (apache with php) and MySQL.

```
networks:  
  my_custom_network:  
    driver: bridge
```

services:

php:

build: .

container_name: php_app

ports:

- "8080:80"

networks:

- my_custom_network

volumes:

- ./app:/var/www/html

depends_on:

- mysql

mysql:

image: mysql:8.0

container_name: mysql_db

restart: always

environment:

MYSQL_ROOT_PASSWORD: rootpass

MYSQL_DATABASE: mydatabase

MYSQL_USER: user

MYSQL_PASSWORD: userpass

networks:

- my_custom_network

ports:

- "3306:3306"

volumes:

- mysql_data:/var/lib/mysql

- ./init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:

mysql_data:

Step 3: Create Dockerfile for PHP

```
FROM php:8.1-apache
```

```
# Install necessary extensions
```

```
RUN docker-php-ext-install mysqli pdo pdo_mysql
```

```
# Enable Apache mod_rewrite
```

```
RUN a2enmod rewrite
```

Step 4: Create .env File (Optional)

Create a .env file for storing database credentials:

```
MYSQL_ROOT_PASSWORD=rootpass
MYSQL_DATABASE=mydatabase
MYSQL_USER=user
MYSQL_PASSWORD=userpass
```

Step 5: Initialize MySQL Database with init.sql

Create init.sql file to prepopulate the database:

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE
);
```

Step 6: Create db.php (Database Connection)

Inside app/, create db.php:

```
<?php
$host = 'mysql';
$dbname = 'mydatabase';
$username = 'user';
$password = 'userpass';

try {
  $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
  $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
  die("Connection failed: " . $e->getMessage());
}
?>
```

Step 7: Create CRUD Operations

1. Create (create.php)

```
<?php
include 'db.php';

if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $name = $_POST['name'];
  $email = $_POST['email'];

  $sql = "INSERT INTO users (name, email) VALUES (?, ?)";
  $stmt = $pdo->prepare($sql);
  $stmt->execute([$name, $email]);

  echo "User added successfully!";
}
```

```
}  
?>  
<form method="post">  
  <input type="text" name="name" placeholder="Name" required>  
  <input type="email" name="email" placeholder="Email" required>  
  <button type="submit">Create User</button>  
</form>
```

2. Read (read.php)

```
<?php  
include 'db.php';  
  
$sql = "SELECT * FROM users";  
$stmt = $pdo->query($sql);  
$users = $stmt->fetchAll(PDO::FETCH_ASSOC);  
  
foreach ($users as $user) {  
  echo "{$user['id']} - {$user['name']} - {$user['email']}<br>";  
}  
?>
```

3. Update (update.php)

```
<?php  
include 'db.php';  
  
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
  $id = $_POST['id'];  
  $name = $_POST['name'];  
  $email = $_POST['email'];  
  
  $sql = "UPDATE users SET name = ?, email = ? WHERE id = ?";  
  $stmt = $pdo->prepare($sql);  
  $stmt->execute([$name, $email, $id]);  
  
  echo "User updated!";  
}  
?>  
<form method="post">  
  <input type="text" name="id" placeholder="User ID" required>  
  <input type="text" name="name" placeholder="New Name" required>  
  <input type="email" name="email" placeholder="New Email" required>  
  <button type="submit">Update User</button>  
</form>
```

4. Delete (delete.php)

```
<?php
include 'db.php';

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $id = $_POST['id'];

    $sql = "DELETE FROM users WHERE id = ?";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([$id]);

    echo "User deleted!";
}
?>
<form method="post">
    <input type="text" name="id" placeholder="User ID" required>
    <button type="submit">Delete User</button>
</form>
```

Step 8: Build and Run the Application

Run the following command to build and start the containers:

```
docker-compose up -d --build
```

Check running containers:

```
docker ps
```

Step 9: Access the Application

- Open <http://localhost:8080/read.php> to view the users.
- Use `create.php`, `update.php`, and `delete.php` to perform CRUD operations.

Step 10: Stop and Clean Up Containers

Stop containers:

```
docker-compose down
```

Remove all containers, volumes, and networks:

```
docker-compose down -v
```

Multi-Stage Builds for Optimized Images in Docker

What is Multi-Stage Build?

Multi-stage builds allow you to create smaller, optimized Docker images by using multiple FROM instructions in a single Dockerfile. This helps in:

- ✓ Reducing final image size by removing unnecessary dependencies.
- ✓ Keeping only the essential files required for production.
- ✓ Improving security by minimizing attack surfaces.

Example: Using Multi-Stage Build for a Flask App

Create a simple Flask app and optimize it with a multi-stage build.

Step 1: Setup the Project Structure

```
mkdir flask-multi-stage && cd flask-multi-stage
```

Project structure:

```
flask-multi-stage/  
├── app.py  
├── requirements.txt  
├── Dockerfile  
├── templates/  
│   └── index.html
```

Step 2: Create a Simple Flask App (app.py)

Create and edit app.py:

```
from flask import Flask, render_template  
  
app = Flask(__name__)  
  
@app.route("/")  
def home():  
    return render_template("index.html")  
  
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```

Step 3: Create requirements.txt

This file contains dependencies:

```
flask
```

Step 4: Create a Simple HTML Page (templates/index.html)

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Flask Multi-Stage Build</title>  
</head>
```

```
<body>
  <h1>Welcome to Flask Multi-Stage Build Optimization</h1>
</body>
</html>
```

Step 5: Create an Optimized Dockerfile with Multi-Stage Build
Create and edit Dockerfile:

```
# First Stage: Build Environment
FROM python:3.11 AS builder

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Second Stage: Production Image
FROM python:3.11-slim

WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib/python3.11/site-packages
COPY --from=builder /usr/local/bin /usr/local/bin
COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Step 6: Build and Run the Flask App

```
docker build -t flask-multi-stage .
docker run -p 5000:5000 flask-multi-stage
```

How Does Multi-Stage Build Optimize the Image?

First Stage (builder):

- ✓ Uses python:3.11 (full version) to install dependencies.
- ✓ Keeps the build environment separate.

Second Stage (Final Image):

- ✓ Uses python:3.11-slim, a minimal base image.
- ✓ Copies only necessary files (requirements, app.py, etc.).
- ✓ Reduces final image size (only includes essential files).

Docker Best Practices for Image Optimization and Security

Best Practice 1: Use a Lightweight Base Image

Instead of using a full Python image (python:3.11), use a lighter version (python:3.11-slim) to reduce size.

```
FROM python:3.11-slim
```

Best Practice 2: Use Multi-Stage Builds

Best Practice 3: Avoid Running as Root

By default, Docker containers run as root, which is a security risk.

Solution: Create a non-root user:

Create a non-root user

```
RUN addgroup --system appgroup && adduser --system --group appuser  
USER appuser
```

*Now, even if an attacker gains access, they will have limited privileges.

Best Practice 4: Reduce Layers & Use --no-cache-dir for Dependencies

Each RUN command creates a new image layer. To minimize layers, combine them:

```
RUN apt-get update && apt-get install -y \  
    some-package \  
    && rm -rf /var/lib/apt/lists/*
```

For Python dependencies:

```
RUN pip install --no-cache-dir -r requirements.txt
```

*This prevents unnecessary caching and reduces image size.

Best Practice 5: Do Not Store Secrets in Images

Never hardcode credentials like database passwords or API keys in Dockerfile. Instead, use environment variables.

Example:

```
ENV FLASK_ENV=production
```

or use Docker secrets for sensitive credentials.

Best Practice 6: Use .dockerignore to Exclude Unnecessary Files

Just like .gitignore, .dockerignore prevents unwanted files from being copied into the image.

Create .dockerignore file:

```
__pycache__/  
*.pyc  
.git/  
node_modules/  
.env
```

Best Practice 7: Expose Only Necessary Ports

Only expose ports that are required.

```
EXPOSE 5000
```

Best Practice 8: Use Health Checks

Add a health check to ensure the app is running properly.

Dockerfile

```
HEALTHCHECK CMD curl --fail http://localhost:5000 || exit 1
```

*This helps Kubernetes and Docker to monitor container health.

Best Practice 9: Run Containers with Least Privileges

Avoid running as root inside the container.

Run with a non-root user in docker-compose.yml:

```
services:
  flask_app:
    image: flask-multi-stage
    user: "1001"
```

*This ensures the app does not have root privileges.

Best Practice 10: Regularly Update and Scan Images

Regularly update the base image and scan for vulnerabilities.

Scan your image:

```
docker scout cves
docker scout recommendations
```

*This checks for security vulnerabilities.

Final Optimized and Secure Dockerfile

```
# Use a lightweight base image
FROM python:3.11-slim AS builder

WORKDIR /app

# Install dependencies in a separate stage
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Production stage
FROM python:3.11-slim
WORKDIR /app
```

```
# Use a non-root user
RUN addgroup --system appgroup && adduser --system --group appuser
USER appuser

# Copy only necessary files
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib/python3.11/site-packages
COPY --from=builder /usr/local/bin /usr/local/bin
COPY . .

# Expose only necessary ports
EXPOSE 5000

# Health check
HEALTHCHECK CMD curl --fail http://localhost:5000 || exit 1

CMD ["python", "app.py"]
```

Introduction to Docker Swarm Mode

What is Docker Swarm Mode?

Docker Swarm is Docker's native orchestration tool that enables you to manage multiple containers across a cluster of machines (nodes). It provides high availability, scaling, and load balancing for containerized applications.

Key Features:

- ✓ Clustering: Multiple Docker nodes act as a single virtual system.
- ✓ Service Management: Define services that run across the swarm.
- ✓ Scaling: Increase or decrease the number of container replicas.
- ✓ Load Balancing: Built-in service discovery and routing.
- ✓ Fault Tolerance: Automatic recovery of failed containers.

Creating and Managing Docker Services in Swarm Mode

What is a Docker Service?

A Docker Service is a long-running container controlled by the Swarm Manager. It defines which container should run and how many replicas should exist.

Step 1: Initialize Docker Swarm

Run the following command on the manager node:

```
docker swarm init --advertise-addr <MANAGER-IP>
```

*The output provides a docker swarm join command for worker nodes.

Step 2: Join Worker Nodes

Run the generated join command on worker nodes:

```
docker swarm join --token <TOKEN> <MANAGER-IP>:2377
```

Step 3: Verify Nodes

Check the swarm status:

```
docker node ls
```

Step 4: Deploy a Service

Deploy an Nginx service with 3 replicas:

```
docker service create --name my-nginx --replicas 3 -p 80:80 nginx
```

Step 5: List Running Services

```
docker service ls
```

Step 6: Inspect a Service

```
docker service ps my-nginx
```

Step 7: Remove a Service

```
docker service rm my-nginx
```

Scaling Containers Using Docker Swarm

Why Scale Services?

Scaling ensures that your application can handle increased traffic by adding or removing replicas dynamically.

Step 1: Scale Up the Nginx Service to 5 Replicas

```
docker service scale my-nginx=5
```

Step 2: Verify the Updated Replica Count

```
docker service ls
```

Step 3: Scale Down to 2 Replicas

```
docker service scale my-nginx=2
```

Implementing User and Role-Based Access Control (RBAC)

Why Implement RBAC?

RBAC restricts access to containers, nodes, and services based on roles. This prevents unauthorized changes.

Step 1: Create a User and Assign Roles

```
docker swarm join-token worker
```

*This command lets workers join without administrative access.

Step 2: Assign Roles Using docker config

You can create access control by defining specific user roles for services.

```
docker config create my-config my-config-file.txt
docker service create --name app --config my-config my-image
```

*Only authorized roles can modify my-config.

Step 3: Restrict Access Using Linux Groups

```
usermod -aG docker myuser
```

*Now, myuser has limited permissions to manage Docker.

Step 4: Use Swarm Secrets for Sensitive Data

Instead of hardcoding credentials:

```
docker secret create db_password db_password.txt
docker service create --name mydb --secret db_password my-database
```

Scanning Docker Images for Vulnerabilities

Why Scan Docker Images?

- ✓ Identify security vulnerabilities before deployment.
- ✓ Protect against known exploits.
- ✓ Ensure compliance with security policies.

Step 1: Use Docker's scout Command

```
docker scout cves
docker scout recommendations
```

Step 2: Use Trivy (Recommended)

Trivy is a powerful open-source scanner.

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy my-image
```

Step 3: Use Anchore for Deep Scanning

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock anchore/grype my-image
```

Step 4: Automate Image Scanning in CI/CD

Example using GitHub Actions:

```
jobs:
  security-scan:
    runs-on: ubuntu-latest
    steps:
      - name: Run Trivy Scan
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'my-image'
          format: 'table'
```